

Autonomous Networking

Networks that observe, decide, and adapt

Gaia Maselli

Department of Computer Science
Sapienza University of Rome

A.Y. 2026 – 2027

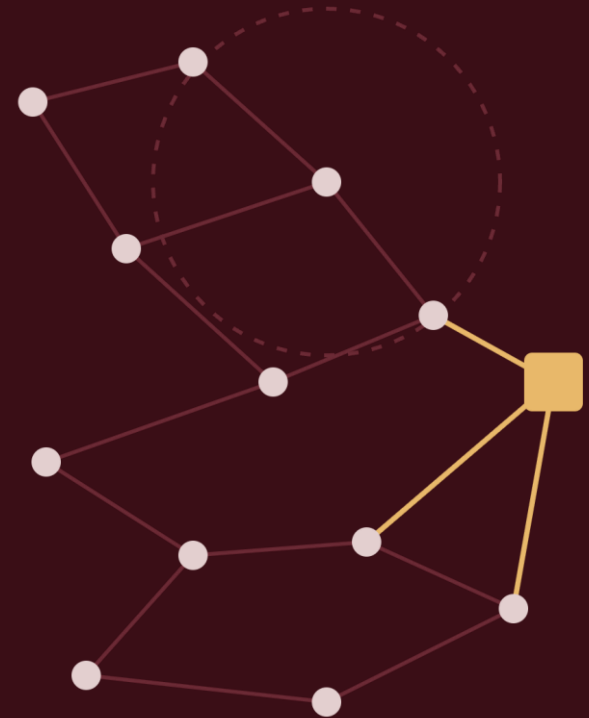


SAPIENZA
UNIVERSITÀ DI ROMA



Wireless Sensor Networks

Characteristics, MAC protocols from CSMA/CA to S-MAC, and routing



A QUESTION TO START

A sensor node on a bridge or in a vineyard has to work for months or years on a battery that nobody will replace.

What should it spend that energy on?

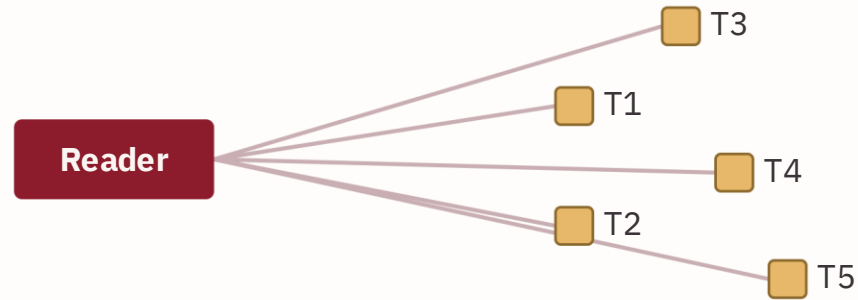
Keep it in mind: we answer it when we reach the MAC layer.

Today's plan

01	Wireless sensor networks	Architecture, nodes, requirements, the wireless channel
02	Energy	Why the radio dominates and where the MAC wastes it
03	MAC protocols and CSMA/CA	802.11 DCF, hidden and exposed terminals, RTS/CTS, NAV
04	S-MAC in depth	Listen/sleep, synchronization, overhearing, evaluation
05	Routing	Proactive (DSDV), reactive (flooding, DSR, AODV), geographic

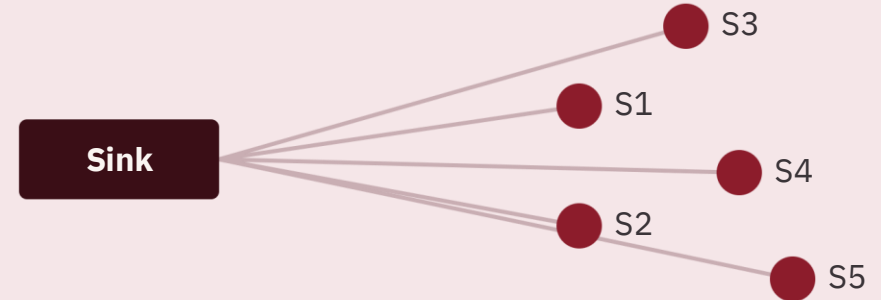
From RFID to sensor networks

RFID NETWORK



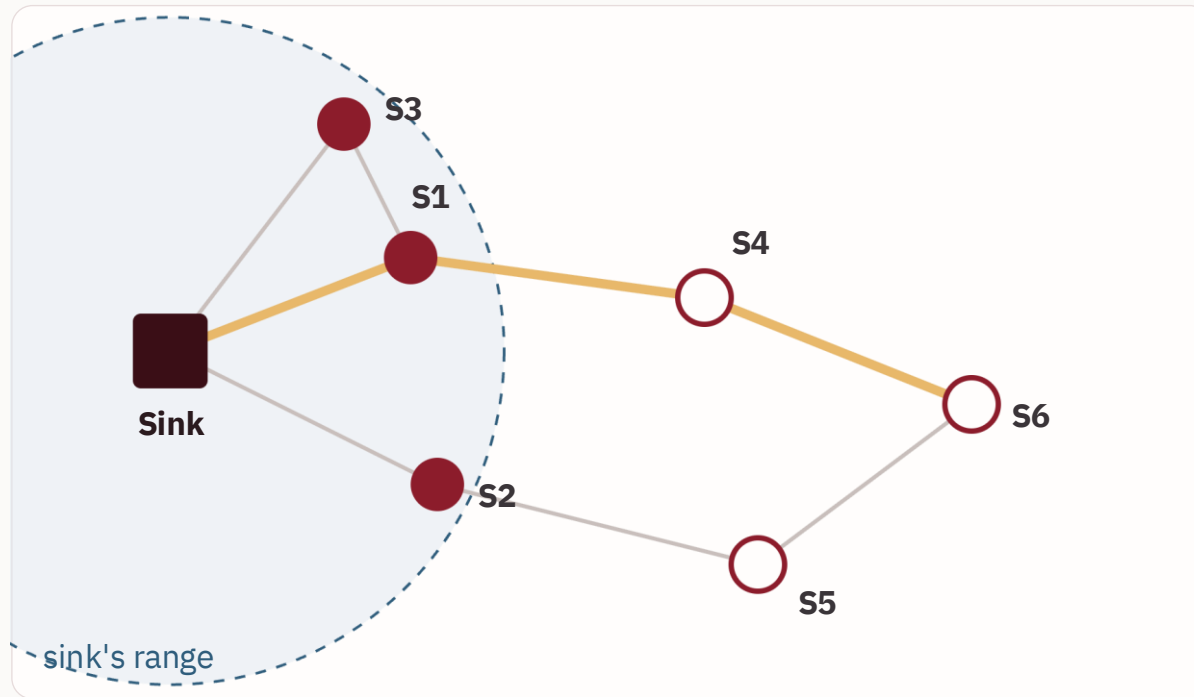
- Tags only send their ID (typically 96 bits, at most 256)
- They answer the reader, they never talk on their own
- Single-hop star around the reader

SENSOR NETWORK



- Battery on board, continuous sensing, local computation
- Carrier sense and **spontaneous** transmission, no backscattering
- So they need a real MAC protocol

Beyond the star: multi-hop



Limited range

Nodes beyond the sink's range (S4–S6) cannot reach it directly

Neighbors relay

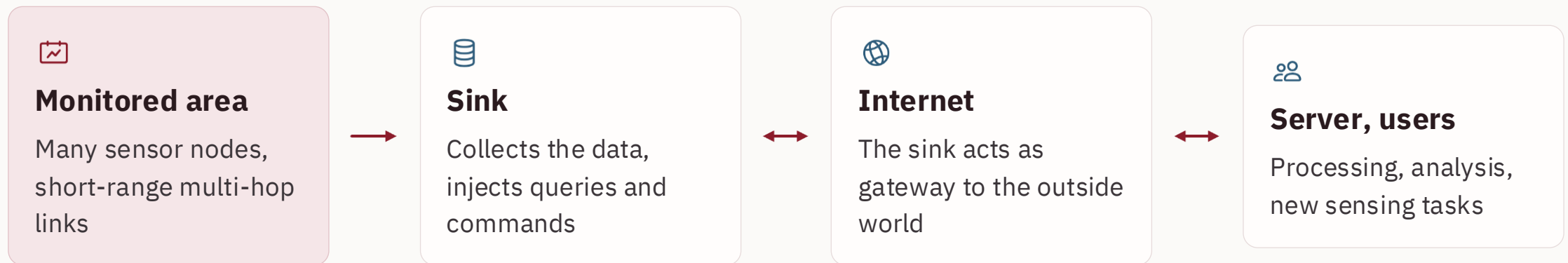
Every node is both a source and a router, e.g. $S6 \rightarrow S4 \rightarrow S1 \rightarrow \text{sink}$

New questions

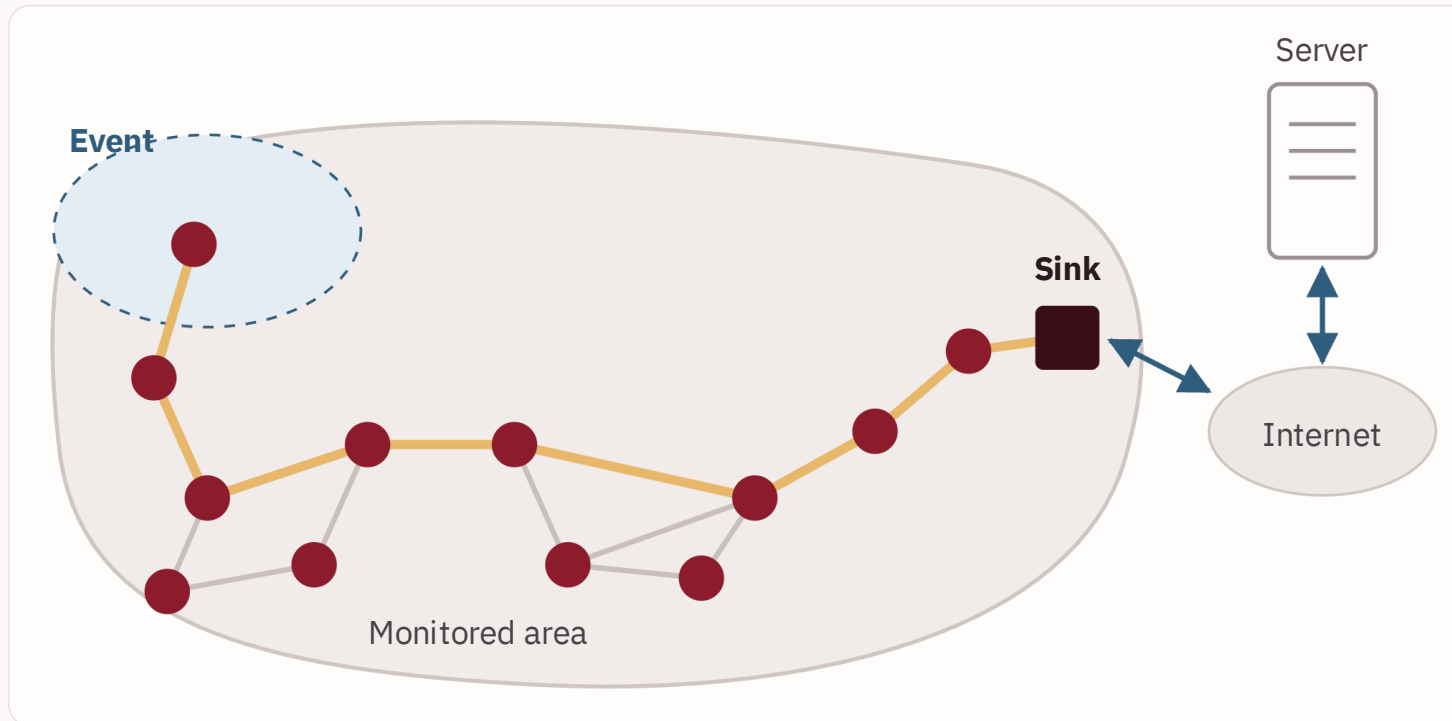
Relays must be awake when neighbors talk (MAC) and must know where to forward (routing)

What is a WSN?

Distributed devices that monitor environmental conditions and send their data, often over multiple hops, to a central node: the **sink**.



Architecture and data gathering



Sense

Nodes near the event detect it

Forward

Data hops node by node toward the sink

Deliver

The sink is the gateway to the Internet and the server

Applications at a glance



Environment

Wildlife, vineyards, forest fires, floods



Structures

Bridges, historical buildings, long-term damage detection



Health care

In-hospital and at-home monitoring, fall detection



Industry

Networked control: closing the loop over a WSN



Security

Hazardous substances, surveillance, early warning



Sea and sky

Underwater WSNs, drone networks

CASE STUDY

Golden Gate Bridge (Stanford, 2005)

64 **46**

nodes

hops

Synchronous monitoring of ambient vibrations, installed without disrupting the bridge's operation

Who takes part, and how nodes are deployed

ROLES



Sources of data

Measure physical quantities and report them "somewhere"



Sinks of data

Interested in receiving the data produced by the WSN



Actors, actuators

Control some device based on the data they receive

DEPLOYMENT OPTIONS

Random

Dropped, e.g. from an aircraft; a uniform random distribution over the area is usually assumed

Regular

Planned and fixed; not necessarily a geometric grid, though often assumed so

Mobile

Moved passively (wind, water) or actively, to fix coverage gaps or seek interesting areas

Anatomy of a sensor node



Sensing unit

Sensors, signal conditioning and ADC; a model maps raw values to physical units



CPU and memory

Low-power microcontroller, a few KB of RAM, flash for code and data



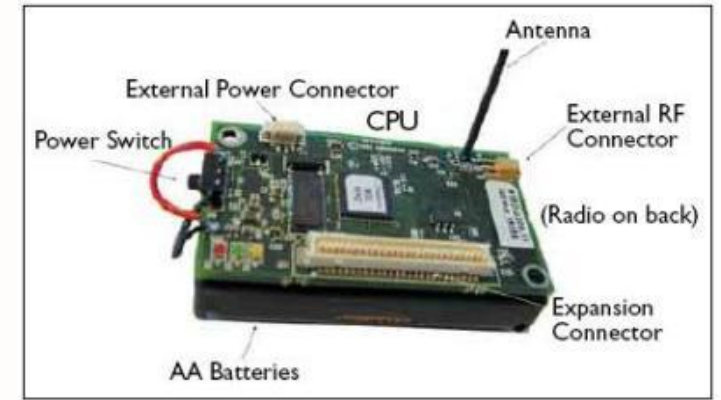
Transceiver

Antenna and low-power RF radio: the component the MAC protocol controls



Power unit

Batteries, sometimes energy harvesting; often impossible to replace



A Mica2 mote, a classic research platform (S-MAC was first implemented on Mica motes)

Software: **TinyOS**, an event-driven OS from UC Berkeley, programmed in **nesC**

WSN vs. conventional wireless networks

Aspect	Conventional networks	Wireless sensor network
Design	General purpose, many applications	One application, or a bouquet of them
Main concern	Throughput, latency, per-user QoS	Energy: network lifetime first
Traffic	Arbitrary pairs, sustained flows	Many-to-one, low rate, event-driven
Environment	Controlled, mild, often infrastructure	Unattended, harsh, even hostile
Access	Easily accessible	Physical access difficult or undesirable
Management	Global knowledge, centralized	Localized decisions, no central entity
Scale	Tens of devices	Hundreds to thousands, often dense

Requirements and design issues

Scalability

Many nodes; performance must not degrade as nodes are added

Density range

From sparse to very dense, depending on the application

Limited resources

Little energy, low cost, small size; maybe no global ID

Dynamic topology

Failures, sleep cycles, mobility, newly deployed nodes

Fault tolerance

Robust to node failure, depleted batteries, physical damage

Lifetime

Fulfil the task as long as possible; definition depends on the application

Programmability

Nodes may need re-programming once in the field

Maintainability

Self-monitoring; adapt to changes and to new resources

Mechanisms that meet the requirements

TYPICAL MECHANISMS

-  **Multi-hop communication**
Short links save energy and get around obstacles
-  **Energy-efficient operation**
For communication, computation, sensing, actuation
-  **Self-configuration**
No operator: nodes find neighbors and organize
-  **In-network processing**
Nodes collaborate and pre-process data on the way

DESIGN PRINCIPLES

Data-centric networking

Focus on the data, not on node identifiers

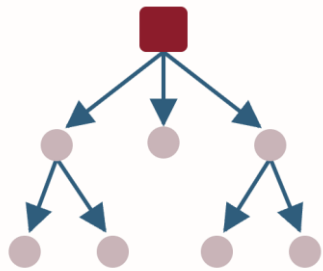
Locality

Do things on the node or among nearby neighbors

Exploit tradeoffs

Energy vs. accuracy, energy vs. **latency**: the S-MAC knob

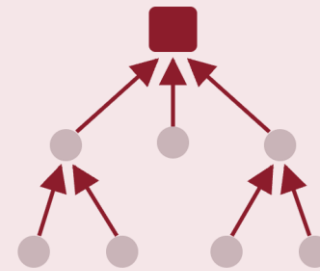
Communication patterns



Dissemination

One to all: sink to every node

Queries and interests, program updates, control packets



Convergecast

Many to one: nodes to the sink

Collection of sensed data, network management and monitoring

For the MAC: traffic is **light, bursty and correlated**; most of the time, nobody is transmitting.

Nodes near the sink relay more traffic

Two constraints shape everything



Wireless signal

Shared, short-range, unreliable: attenuation, multi-path, interference, errors

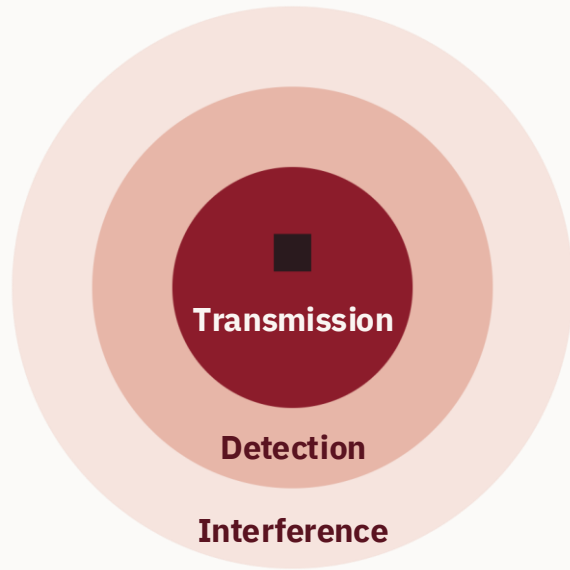


Battery powered

Finite and rarely replaced: every bit sent, received or waited for costs lifetime

The MAC layer sits where the two meet: it decides **when the radio is on**.

The wireless channel



Attenuation

Signal strength drops quickly with distance:
short links, many hops

Multi-path

Reflections on walls, ground and objects:
copies arrive by several paths

Interference

Other senders on the same band, reaching
farther than you can decode

Errors

Low signal-to-noise ratio: the frame cannot
be recovered and is resent

ENERGY

Listening costs as much as talking

TRANSMIT

17.4

mA at 0 dBm

RECEIVE, LISTEN

18.8

mA, even if nobody talks

IDLE

0.43

mA, oscillator on

SLEEP

0.02

mA, power-down

The only cheap radio is a radio that is **off**: roughly three orders of magnitude between listening and sleeping.

Where the MAC layer wastes energy

THE BIG ONE

Idle listening

Listening to an idle channel in order to receive possible traffic. With light, event-driven traffic it dominates the budget.

Collisions

Overlapping packets are discarded and must be retransmitted

Overhearing

Receiving packets destined to other nodes

Control overhead

Control packets that carry no data, often as big as the data

Overemitting

Sending when the destination is not ready to receive

What we ask of a sensor MAC

The MAC decides when the radio **sends**, when it **listens** and when it **sleeps**: the most energy-relevant decision a node makes.

PRIMARY

Energy efficiency

Prolong network lifetime

Scalability and adaptivity

Handle changes in size, density and topology quickly

SECONDARY, DESIRABLE

Latency

Throughput

Bandwidth utilization

Fairness

Two families of MAC protocols

Contention-based

Nodes sense the carrier and compete on demand

Pro · scalable, no central authority

Con · idle listening, collisions

Con · hidden and exposed terminals

CSMA/CA (802.11, 802.15.4) · **S-MAC** · T-MAC · B-MAC · X-MAC

Schedule-based

A schedule says when each node may transmit

Pro · energy efficient: known sleep times

Pro · no collisions by design

Con · synchronization, central authority

TDMA · TRAMA · LMAC · hybrid: Z-MAC

CSMA/CA in IEEE 802.11 DCF

Why "avoidance"? A radio cannot detect collisions while transmitting, so it must try hard to avoid them.

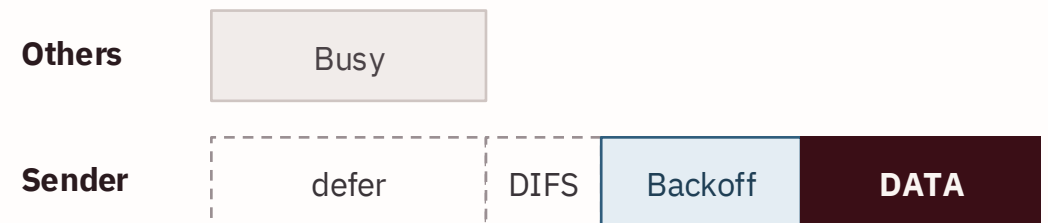
Channel idle

- Idle for a full DIFS: transmit at once, with no backoff
- The receiver checks the CRC and sends an ACK after a SIFS
- ACK received: success, CW goes back to CW_{min}



Channel busy

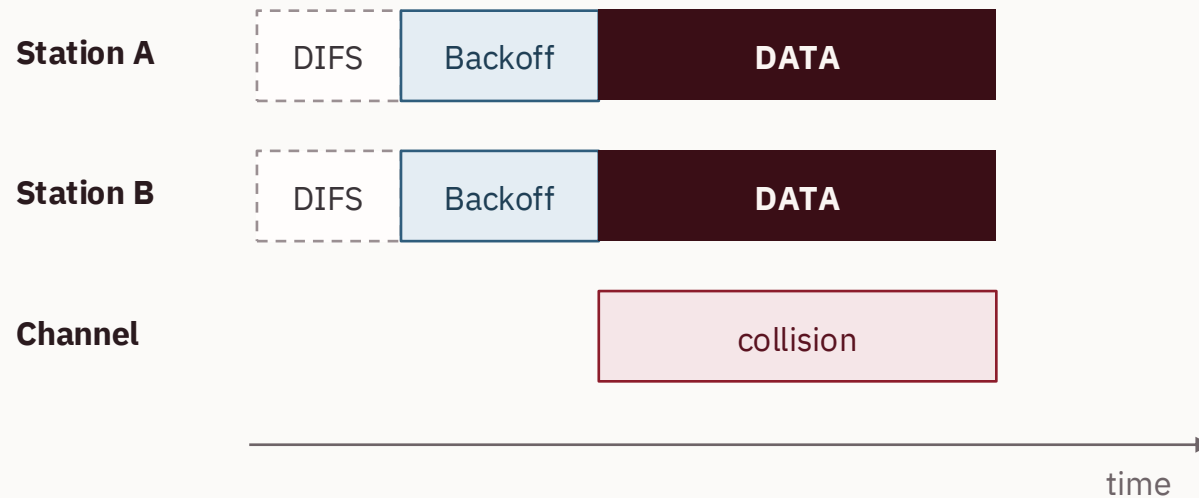
- Defer: keep sensing until the ongoing transmission ends
- Wait a DIFS, then draw a random backoff and count down while the channel is idle
- Counter at zero: transmit



Either way, **no ACK means a collision**: CW doubles and the frame is retried.

Carrier sensing and its limits

TWO STATIONS, SAME SLOT



Both stations draw the same backoff value, reach zero in the same slot and transmit together: neither sees a busy channel. The same happens if B's counter expires up to one slot after A's: A's signal has not been detected yet.

WHAT CARRIER SENSE REALLY DOES

- Clear channel assessment (CCA): energy above a threshold, a valid preamble, or both
- The slot time covers detection time, radio turnaround and propagation; backoff is counted in slots
- A frame that started less than one slot ago is invisible: the vulnerable window
- Avoidance is probabilistic: collisions become rare, not impossible

A collision is never detected: the sender learns of it only from the **missing ACK**.

Priorities and backoff

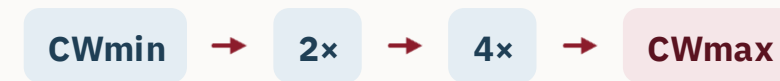
INTER-FRAME SPACES

IFS	Duration	Used for
SIFS	Shortest	ACK, CTS, poll response
PIFS	SIFS + 1 slot	Point coordinator (PCF)
DIFS	SIFS + 2 slots	Asynchronous data

Shorter wait means higher priority: a response always wins over new data.

BACKOFF CLOCK

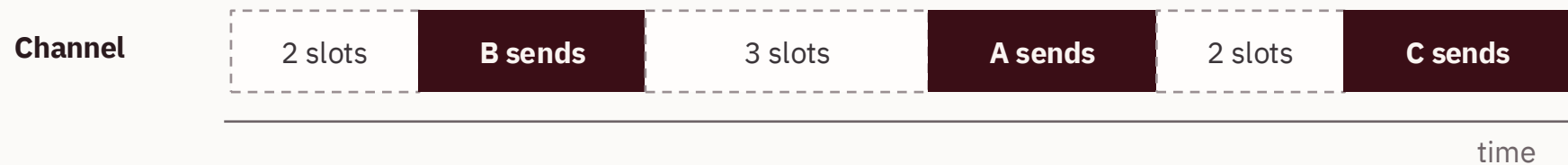
- Random value in $[0, CW-1]$ slots
- All contenders count down; first to reach 0 transmits
- The others freeze, then resume in the next contention period



Binary exponential backoff: CW doubles after each failure, resets after success

Backoff by example

THREE STATIONS DRAW A = 5, B = 2, C = 7 SLOTS (CW = 8)



REMAINING COUNTER AFTER n IDLE SLOTS

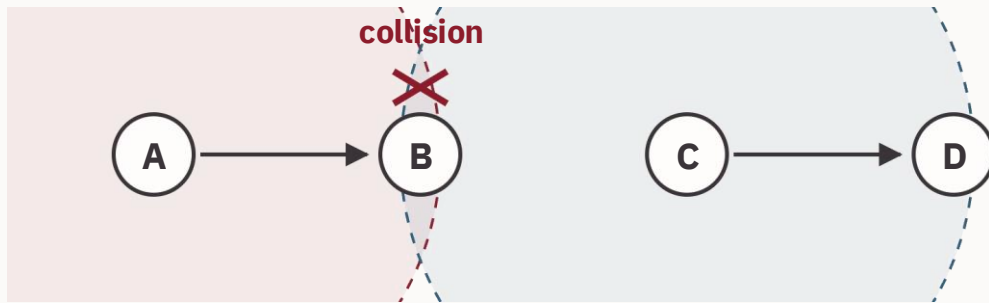
Station	Draw	After 2	After 5	After 7
A	5	3 (frozen)	0: sends	–
B	2	0: sends	–	–
C	7	5 (frozen)	2 (frozen)	0: sends

- Freezing keeps the time already waited: A and C resume where they stopped
- The random draws decide the order of access, with no coordination
- Two equal draws reach zero together and collide

Same draw, same slot: both transmit, no ACK comes back, and both **double their CW** (next slide).

Hidden and exposed terminals

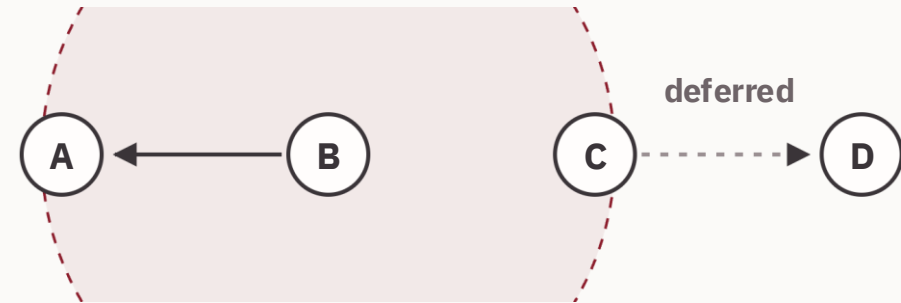
Hidden terminal



A and C cannot hear each other: carrier sense fails, and C's frame to D collides with A's frame at B.

RTS/CTS fix: C overhears B's CTS and inhibits its transmitter.

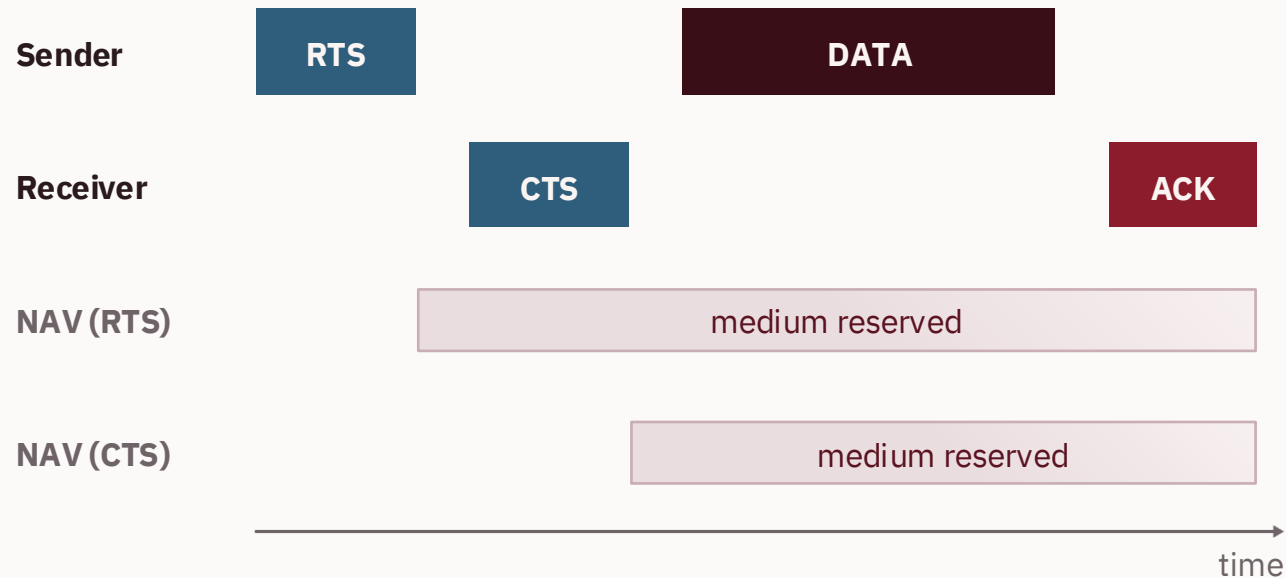
Exposed terminal



C hears B's transmission to A and postpones its frame to D, although it would not collide at A.

RTS/CTS fix: C hears B's RTS but not A's CTS, so it may transmit.

RTS/CTS and virtual carrier sense

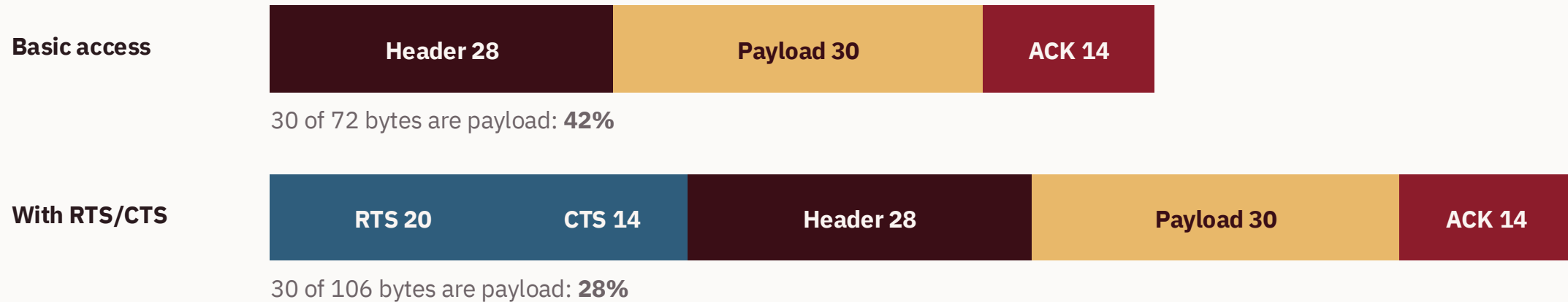


- Every frame carries a **duration** field
- Neighbors set their NAV and count it down
- $NAV > 0$, or carrier sensed: the medium is busy
- Collisions now hit short RTS frames, not data

In S-MAC the NAV becomes a **sleep timer**.

The price of the handshake

BYTES ON THE AIR FOR ONE 30-BYTE SENSOR READING (802.11 MAC FRAMES)



- Control frames cost time as well as bytes: each one has its own PHY preamble and needs a radio turnaround (SIFS)
- That is why 802.11 uses RTS/CTS only above an RTS threshold; a sensor reading almost never reaches it

Sizes: RTS 20 B, CTS 14 B, ACK 14 B, data header and FCS about 28 B. PHY preamble and header are not counted.

Why plain CSMA/CA does not fit sensors

The radio is always on

Stations listen between frames: at low load, **idle listening** dominates

Everyone decodes everything

Neighbors receive frames not meant for them: **overhearing**

Heavy handshakes

RTS, CTS, ACK are comparable to a short sensor payload: **control overhead**

The wrong objectives

Tuned for per-node fairness and throughput, not for network lifetime

Keep the collision avoidance, add **coordinated sleep**: that is S-MAC.

MAC PROTOCOLS

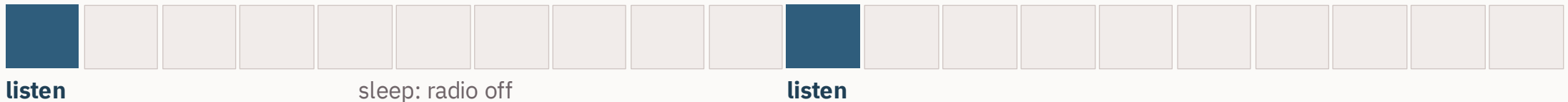
The central idea: duty cycling

Turn the radio off, but stay reachable: the node sleeps most of the time and wakes up in short, agreed windows.

Always-on radio 100% awake: listening, receiving, or idle-listening



Duty-cycled radio 10% awake: short listen windows, long sleep periods



Listening costs as much as receiving, so the only real saving is to switch the radio off. **The price: a sleeping node cannot hear, so neighbors must agree on when to be awake.**

S-MAC: a MAC designed for sensors

Ye, Heidemann, Estrin · IEEE INFOCOM 2002 · extended in IEEE/ACM Transactions on Networking, 2004

1

Periodic listen and sleep

Targets idle listening

2

Collision and overhearing avoidance

Targets collisions, overhearing

3

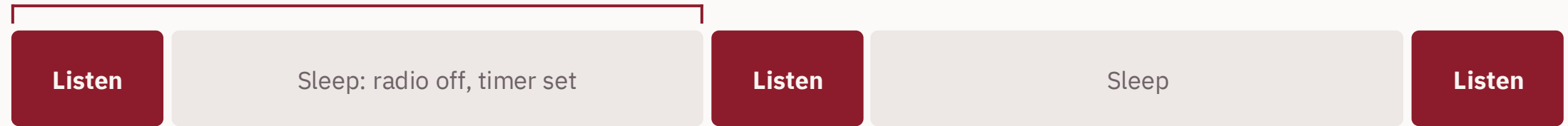
Message passing

Targets control overhead, long messages

The deal: give up some **latency** and **per-node fairness** to gain **energy**.

Periodic listen and sleep

One frame = listen + sleep



Duty cycle

$T_{\text{listen}} / T_{\text{frame}}$

Listen 200 ms, sleep 2 s: about 10%,
radio on one tenth of the time

Fixed and shared

Listen interval fixed a priori, set by the
application; neighbors are awake together

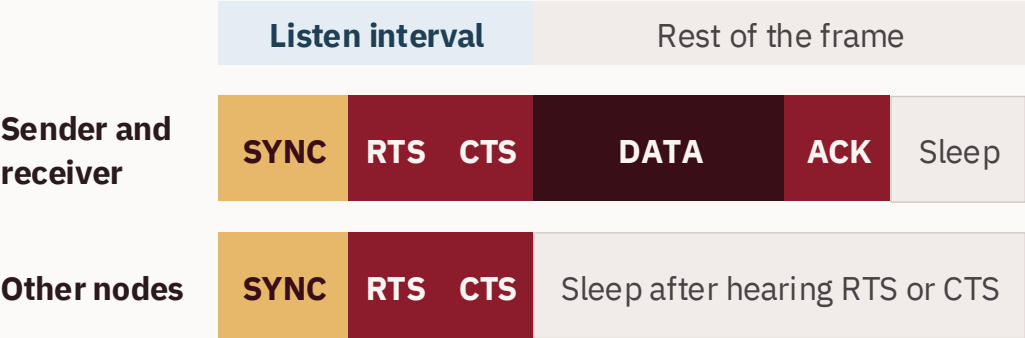
Loosely synchronized

Relative timestamps; listen time much
longer than clock drift

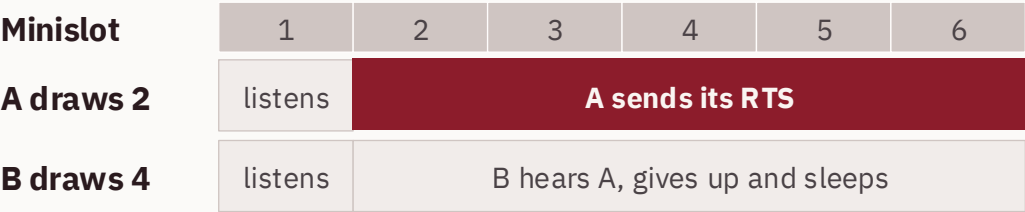
Choosing and spreading schedules

- 1. Listen for a SYNC for some time before choosing
- 2. Nothing heard: pick a random schedule and broadcast it. The node is a **synchronizer**
- 3. A SYNC heard first: adopt it, rebroadcast later. The node is a **follower**
- 4. A different schedule heard later: adopt both. The node is a **border node**
- 5. Periodic SYNCs correct drift and let new nodes join

ONE S-MAC FRAME



ZOOM ON THE RTS/CTS PART: 6 MINISLOTS



- **Minislots:** each node picks one at random and sends only if nobody started before it
- **Why:** nodes waking together would otherwise all send at once and collide

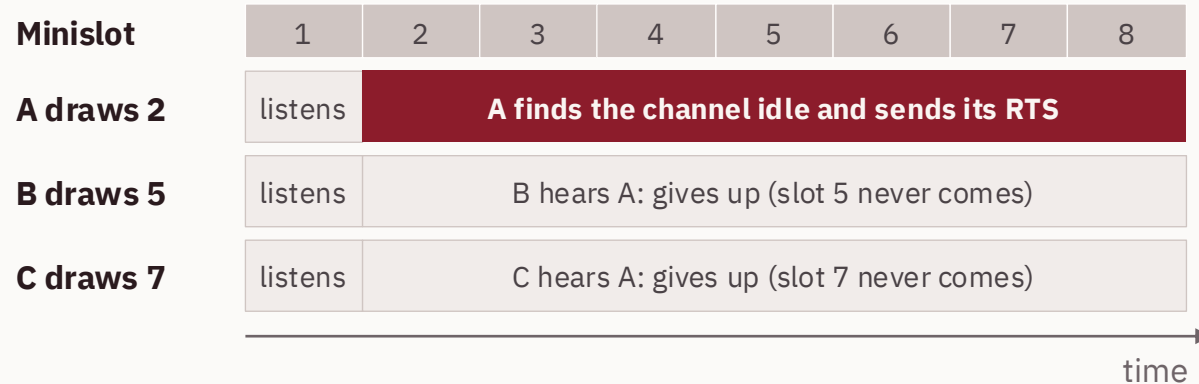
Minislots: avoiding simultaneous sends

WITHOUT MINISLOTS: EVERYONE SENDS AT ONCE

A, B, C

three RTS at the same time: collision, all lost

WITH MINISLOTS: EACH NODE WAITS FOR ITS OWN SLOT



HOW IT WORKS

- **Slices of time:** the listen interval is cut into many short slots, the minislots
- **Random draw:** a node that wants to send picks one minislot at random and listens until then
- **Idle at its slot:** it sends. Busy: it gives up and sleeps, then retries in the next frame
- **Used twice per frame:** in the SYNC part and in the RTS/CTS part

The smallest draw wins. If two nodes draw the same minislot they still collide: the draw makes it unlikely, not impossible.

Keeping schedules in sync

SYNC PACKET

Sender node ID	Next sleep time
----------------	-----------------

Synchronization period

How often a node sends a SYNC; receivers adjust their timers as soon as it arrives

Initial sync

A joining node listens for SYNCs and adopts a neighbor's schedule

Periodic sync

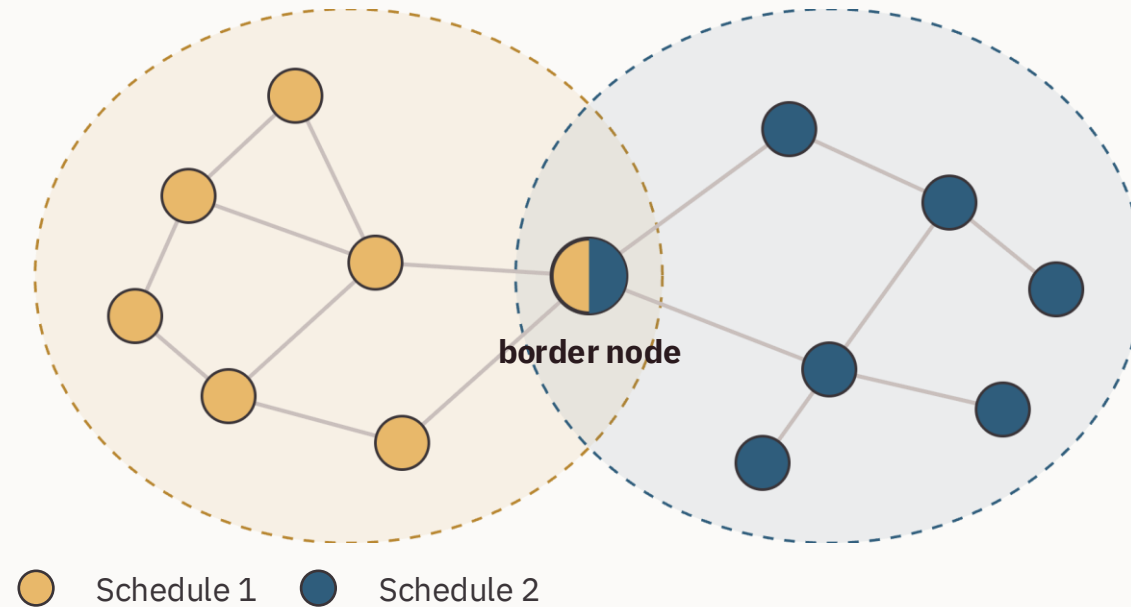
SYNCs keep flowing to compensate for clock drift

SCHEDULE TABLE OF NODE C

Neighbor	Schedule
A	Schedule 1
B	Schedule 1
E	Schedule 2

Entries are filled as SYNC packets arrive. Two schedules in the table: C is a **border node**.

Virtual clusters and border nodes



Shared timing, not hierarchy

Nodes on the same schedule form a virtual cluster: no cluster head, no inter-cluster protocol

Border nodes

Follow both schedules, send broadcasts twice, once per schedule

Neighbor discovery

Nodes periodically listen a whole SYNC period to find neighbors on other schedules

Collision avoidance, borrowed from 802.11

Physical and virtual carrier sense

Duration field in every packet; neighbors set NAV

Unicast: RTS, CTS, DATA, ACK

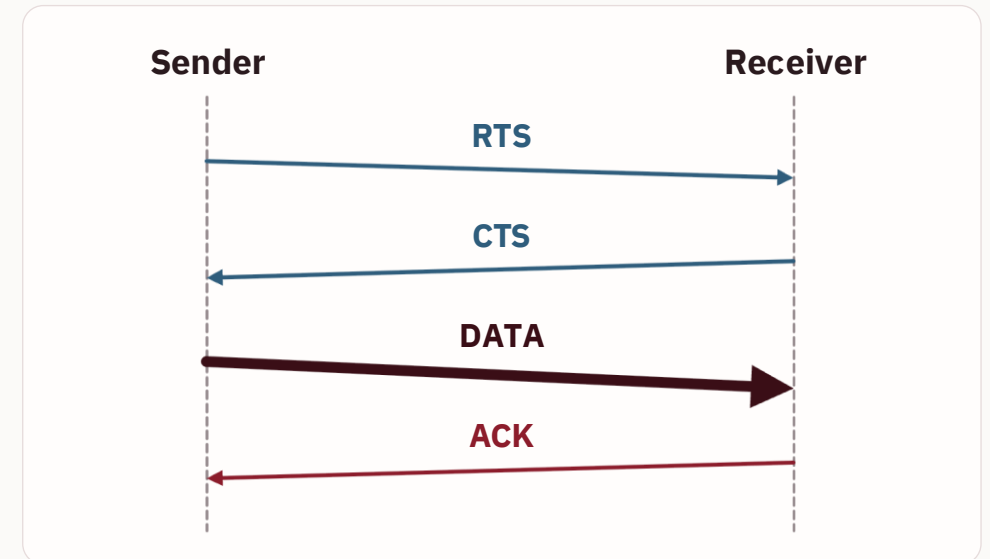
Handles hidden terminals, as in the DCF

Broadcast: no handshake

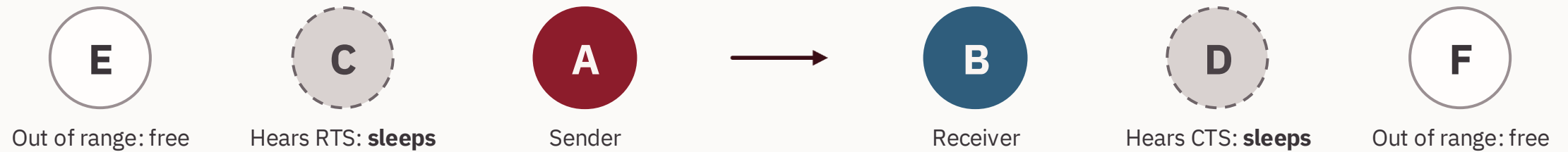
SYNC and broadcasts: no RTS/CTS, no ACK

Lost the contention?

Sleep, and retry when the receiver is free and listening again



Overhearing avoidance



The rule

Hear an RTS or CTS for someone else:
sleep until your NAV expires

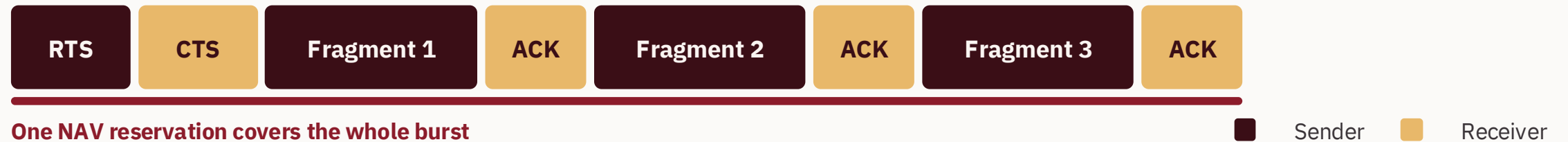
Who sleeps

All immediate neighbors of both sender
and receiver

What it saves

Receiving long DATA frames meant for
others

Message passing for long messages



Fragment and burst

Small fragments are cheap to resend; one RTS/CTS for all of them

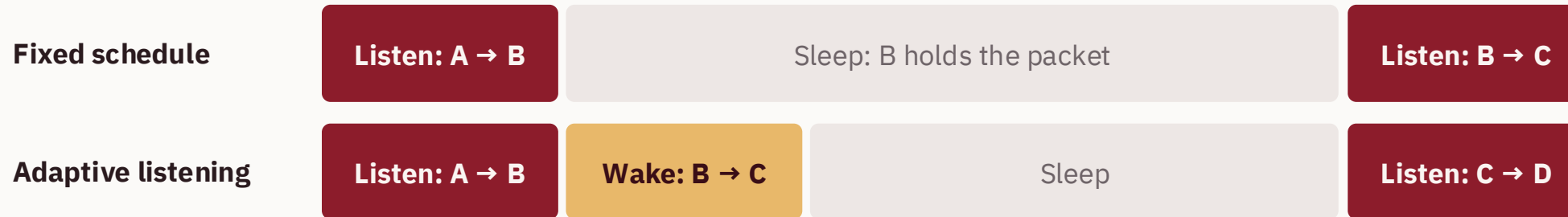
Duration everywhere

Each fragment and ACK carries the remaining time, so waking neighbors go back to sleep

Fairness traded

The message keeps the channel: message latency beats node fairness

Adaptive listening cuts multi-hop delay



Idea

Nodes that overhear an RTS or CTS wake up briefly when that transmission ends

Gain

If one of them is the next hop, the packet moves on at once, not one frame later

Cost

Neighbors not on the path wake up for nothing: a little extra listening

Energy vs. latency: the frame is the knob

ALWAYS ON, 802.11-LIKE

$$D(N) \approx N \cdot (t_{cs} + t_{tx})$$

Carrier sense and transmission per hop;
maximum energy

S-MAC, FIXED SCHEDULE

$$D(N) \approx N \cdot T_{\text{frame}}$$

One hop per frame: sleep delay dominates

WITH ADAPTIVE LISTENING

$$D(N) \approx N \cdot T_{\text{frame}}/2$$

Ideally two hops per frame

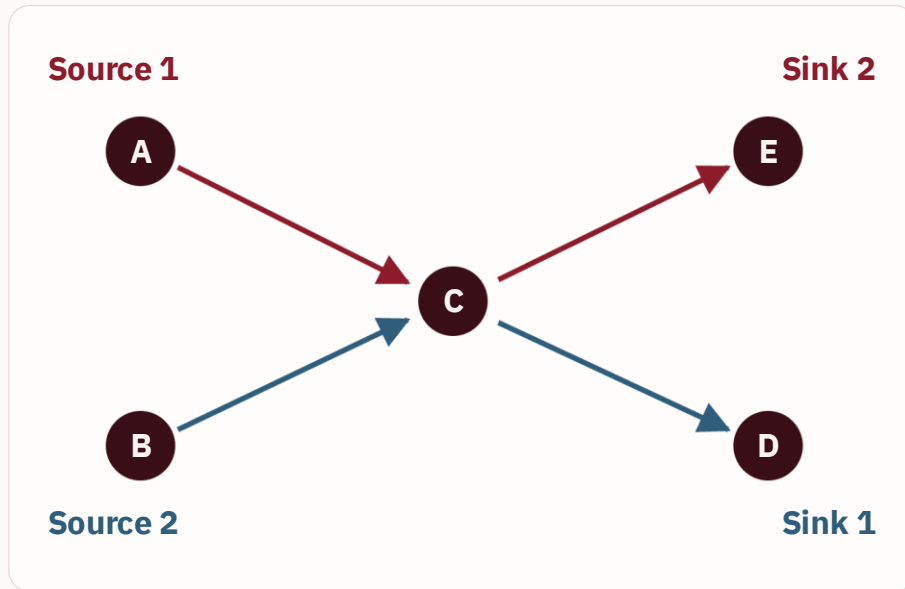
Longer frame, same listen time

Lower duty cycle: less energy, **more delay** on every hop

Back to the bridge

46 hops at $T_{\text{frame}} = 1$ s: about 46 s, or about 23 s with adaptive listening

How S-MAC was evaluated



Topology

Two-hop network: two sources, two sinks, one relay

Fixed amount of data

Each source sends 10 messages of 10 fragments of 40 bytes: 200 data packets in total

Traffic load

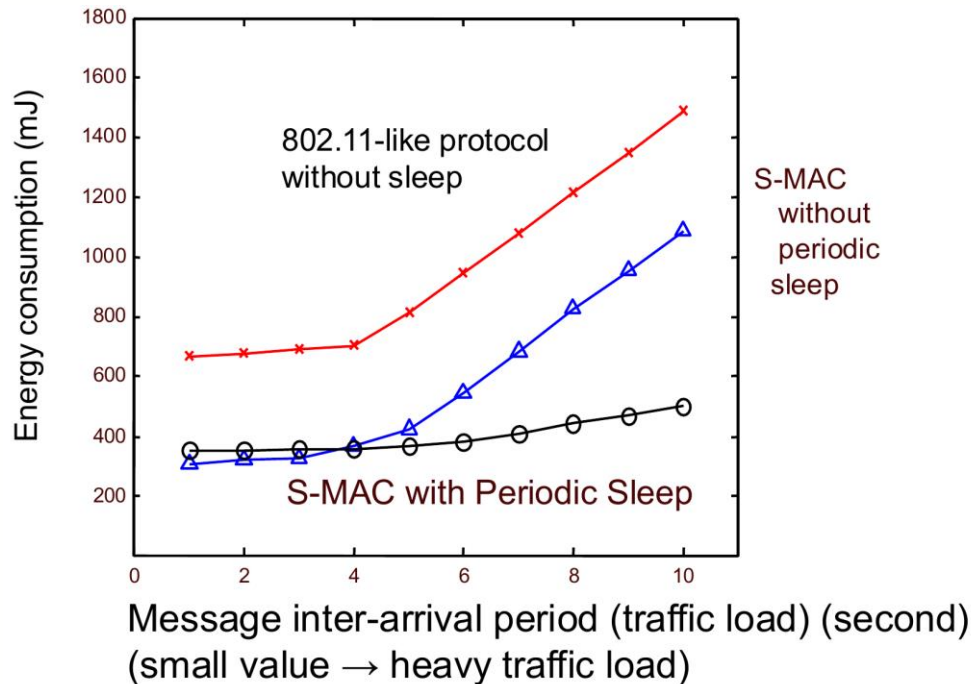
Message inter-arrival period from 1 to 10 s; small values mean heavy load

Metric

Total energy each node spends to deliver the data

What the experiments showed

Average energy in the source nodes A and B



Always below 802.11

From about 2× less energy at heavy load to about 3× less at light load

Heavy load

Idle listening is rare: sleeping saves little; overhearing avoidance and message passing do the work

Light load

Periodic sleep plays the key role: without it, energy grows with idle time

ROUTING

From many hops to the sink

Multi-hop communication needs a routing strategy that keeps energy consumption to a minimum, and so **maximizes network lifetime**.

NETWORK TOPOLOGY

Flat or hierarchical

All nodes play the same role, or some act as cluster heads

HOW NODES ARE IDENTIFIED

An ID, or a position

With positions, the next hop follows from the destination address:
scalable, suited to WSNs

When does the protocol work?



Proactive

Table-driven

Keeps routes up to date at all times, before they are needed

DSDV



Reactive

On demand

Finds a route only when a packet actually needs one

Flooding, gossiping, DSR, AODV



Hybrid

Both

Combines the two behaviors, e.g. proactive nearby, reactive far away

e.g. ZRP



Geographic

Position-based

No explicit tables: the next hop follows from positions

Most forward within r , ...

Proactive routing and DSDV

Table-driven, like wired networks

Every node keeps routes to all destinations, needed or not, and floods updates when the topology changes

Pro · the route is ready: no discovery delay

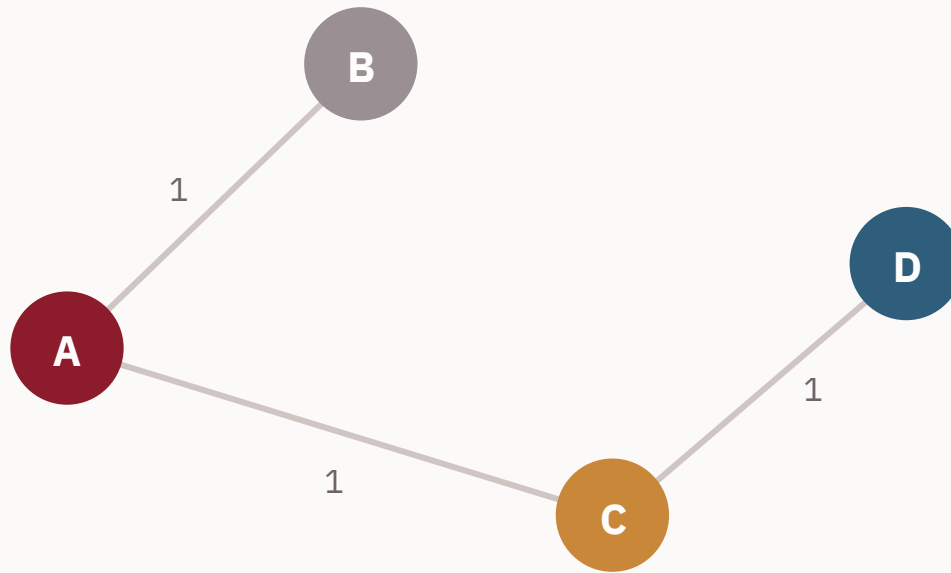
Con · updates cost bandwidth and battery

Con · the information may still be out of date

DSDV: Destination-Sequenced Distance Vector

- Distance vector: distributed Bellman-Ford
- Periodic full updates, incremental updates on changes
- **Sequence number** per entry: higher means fresher, avoids loops
- Unstable route updates are delayed

DSDV: the example network



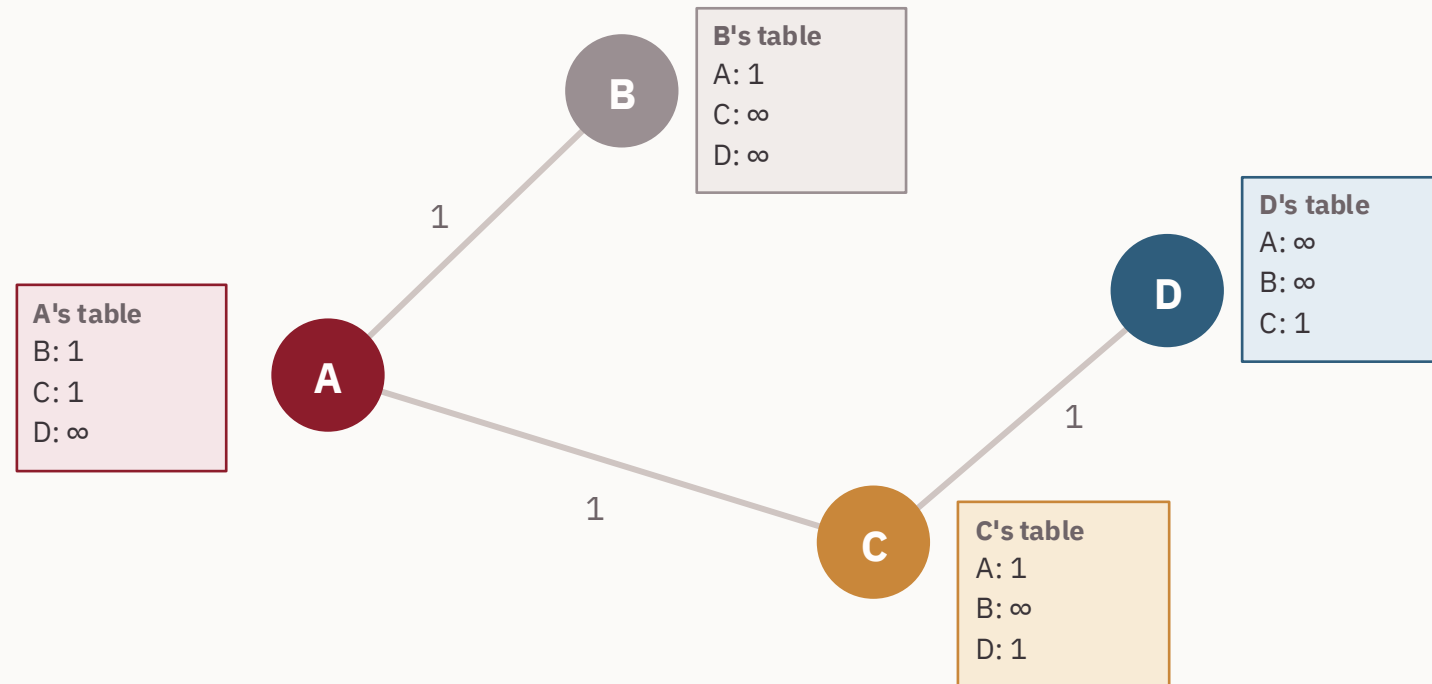
WHAT HAPPENS

- **Four nodes, three links:** A–B, A–C and C–D, each of cost 1 (one hop)
- **Distance vector:** every node tells its neighbors how far it is from each destination
- **Sequence numbers:** every entry also carries one, generated by the destination; higher means fresher (omitted in the figures)

LEGEND

-  table of the node
-  vector just received
- 1** entry updated in this step

DSDV: the starting vectors



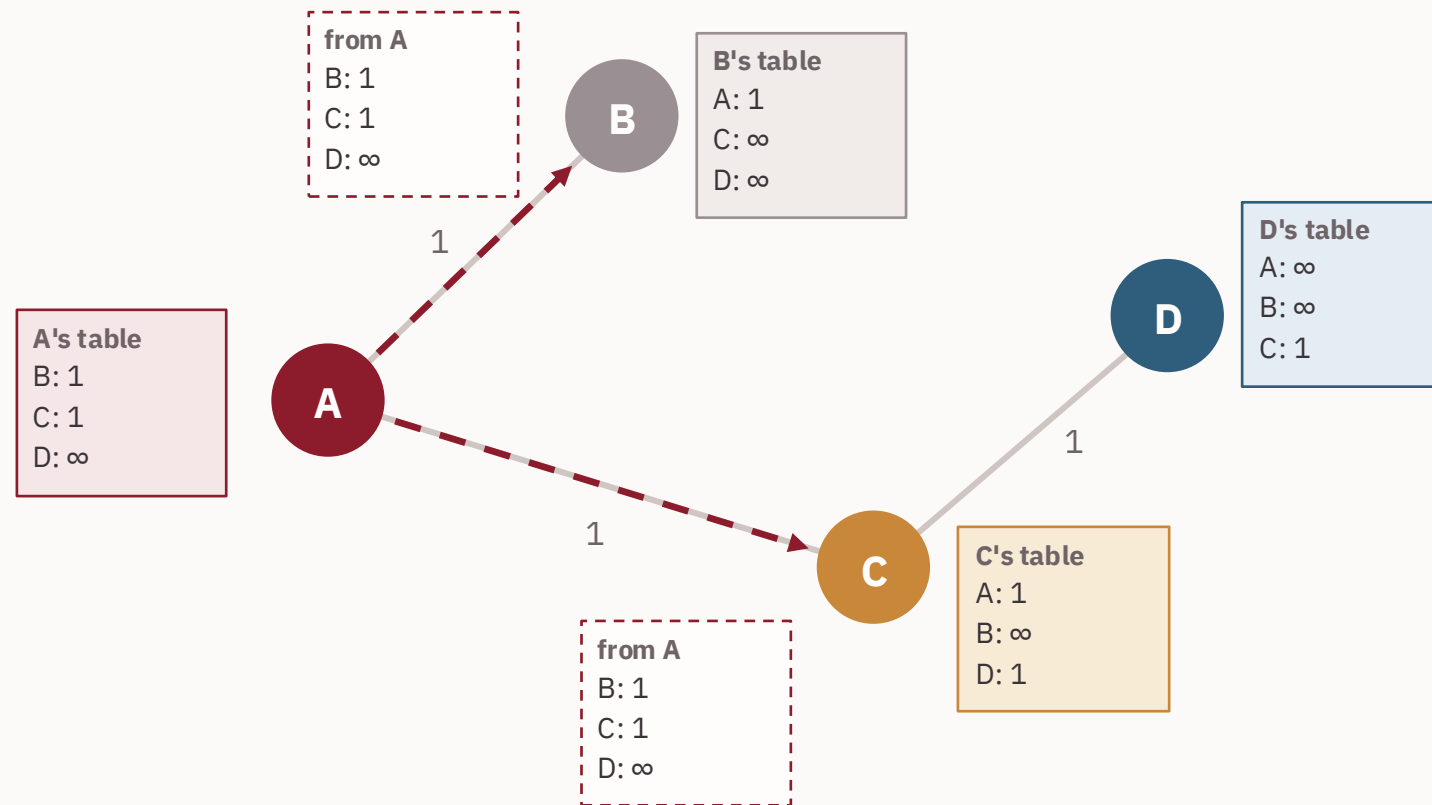
WHAT HAPPENS

- **Each node knows only its neighbors**, at distance 1
- **Every other destination is at infinity**: no route is known yet
- Each entry reads **destination: hops**

LEGEND

-  table of the node
-  vector just received
- 1** entry updated in this step

DSDV: A sends its vector



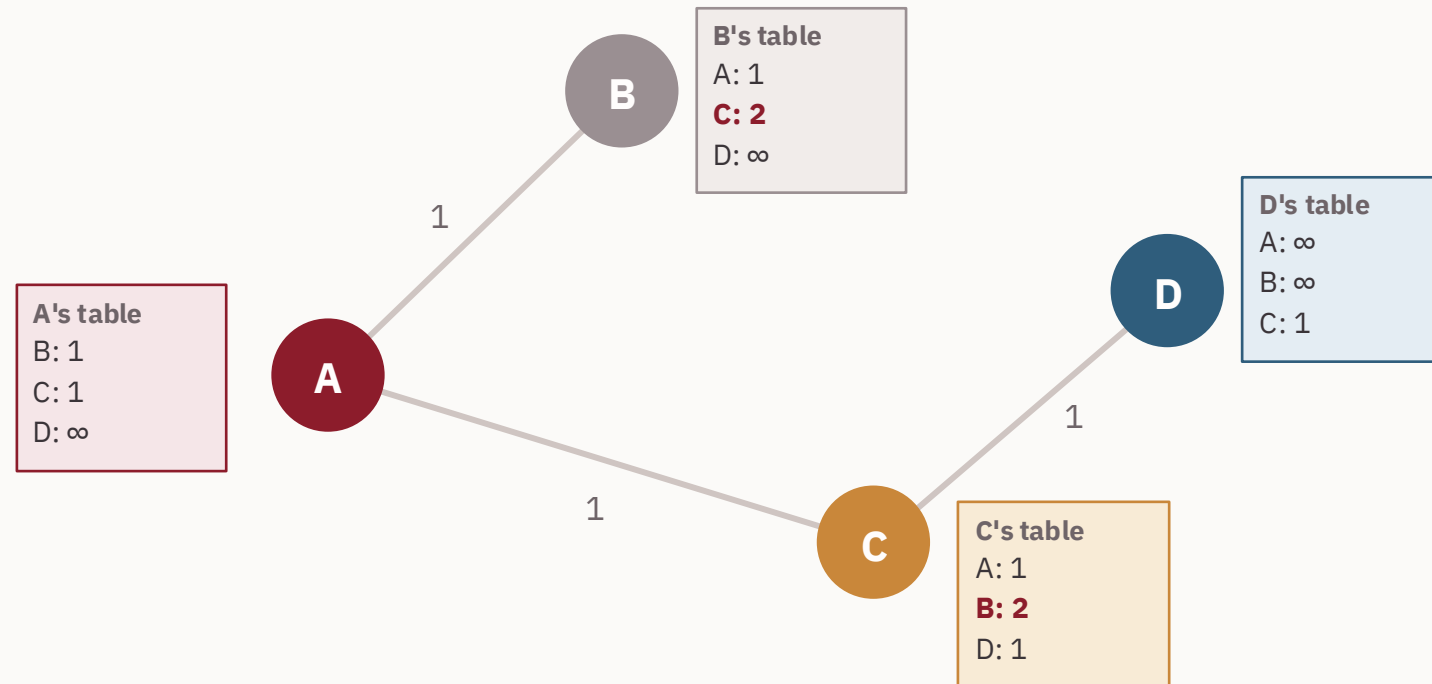
WHAT HAPPENS

- **A sends its vector** (B: 1, C: 1, D: ∞) to its neighbors B and C
- **Receivers add the link cost:** a destination at distance d from A is at $d + 1$ from the receiver
- **Updates are periodic:** every node does the same with its own vector

LEGEND

- table of the node
- vector just received
- 1** entry updated in this step

DSDV: B and C update their tables



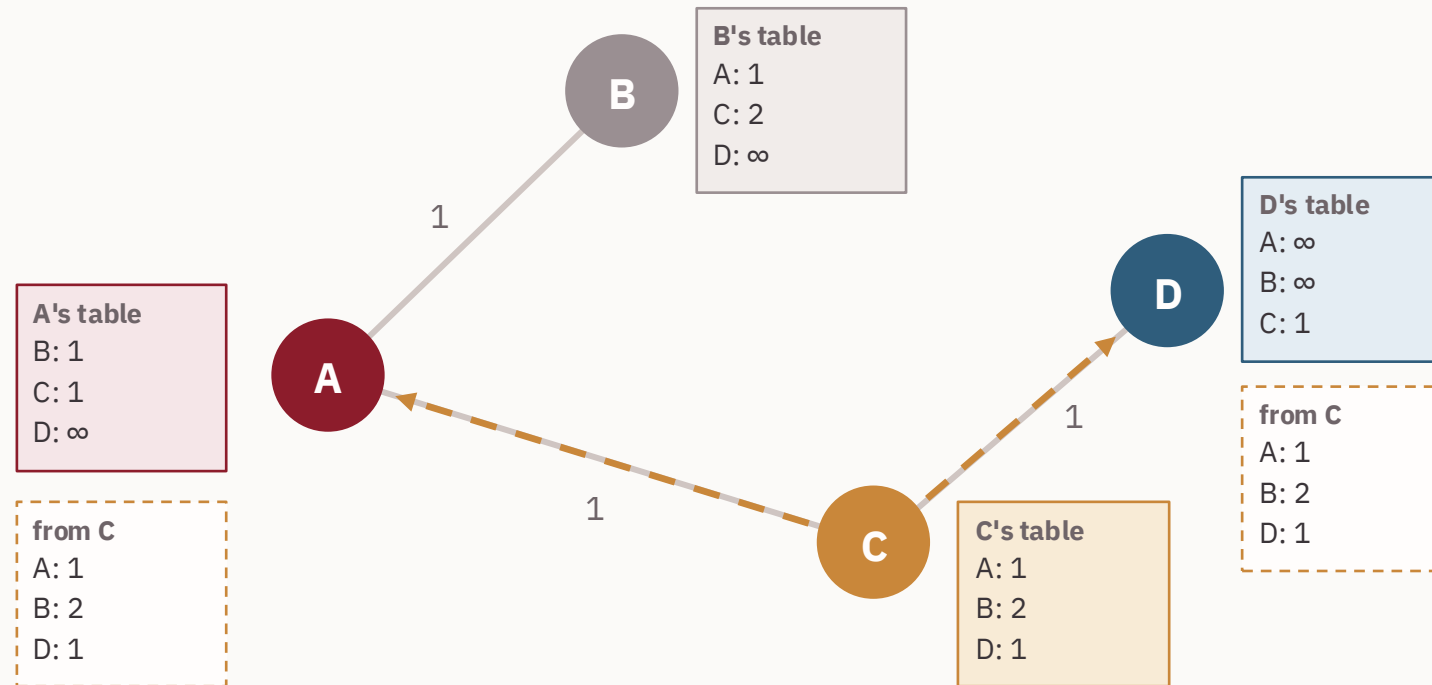
WHAT HAPPENS

- **B learns C at 2:** A reaches C at 1, B reaches A at 1, so C is at $1 + 1 = 2$ through A
- **C learns B at 2** in the same way
- **D: still ∞** for B: A had no news about D yet

LEGEND

- table of the node
- vector just received
- 1** entry updated in this step

DSDV: C sends its vector



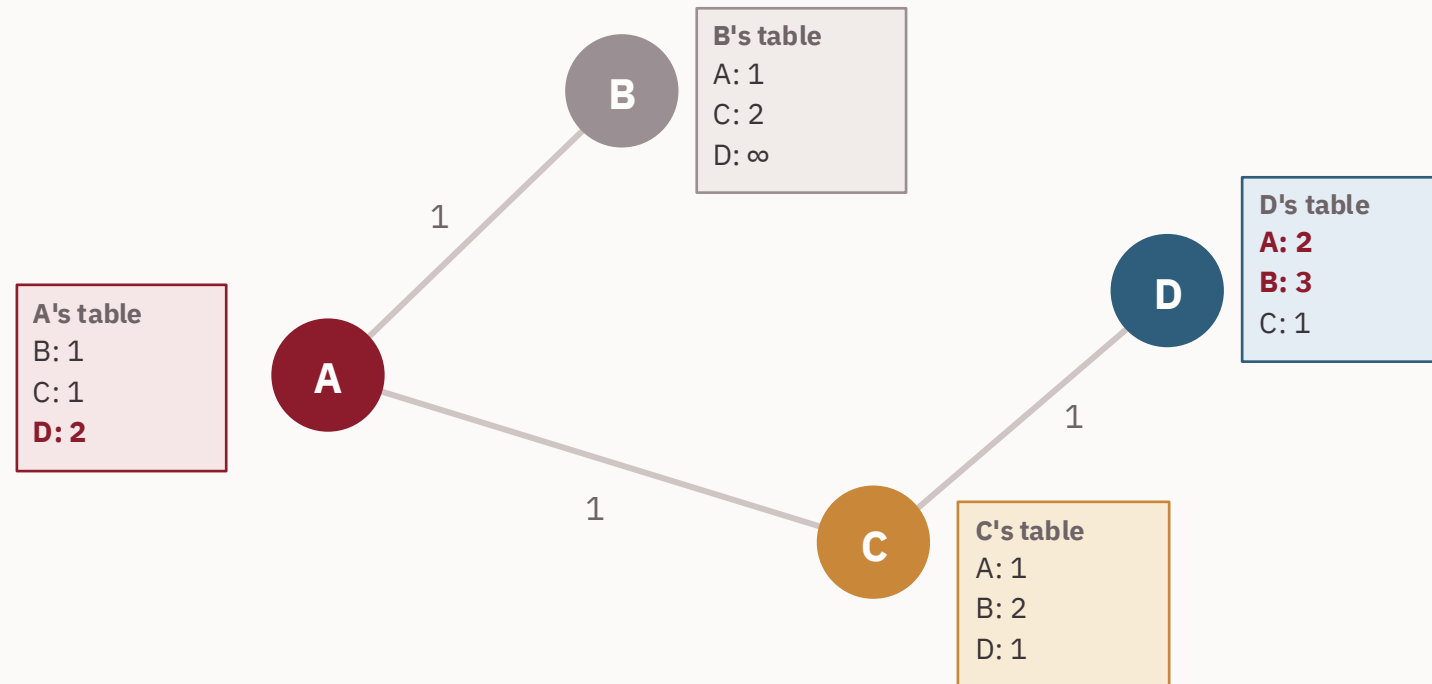
WHAT HAPPENS

- **C has news:** it now sends (A: 1, B: 2, D: 1) to its neighbors A and D
- **Same rule:** each receiver adds 1 to the distances it reads
- **Information travels one hop** per exchange, so far destinations take several rounds

LEGEND

- table of the node
- vector just received
- 1** entry updated in this step

DSDV: A and D update



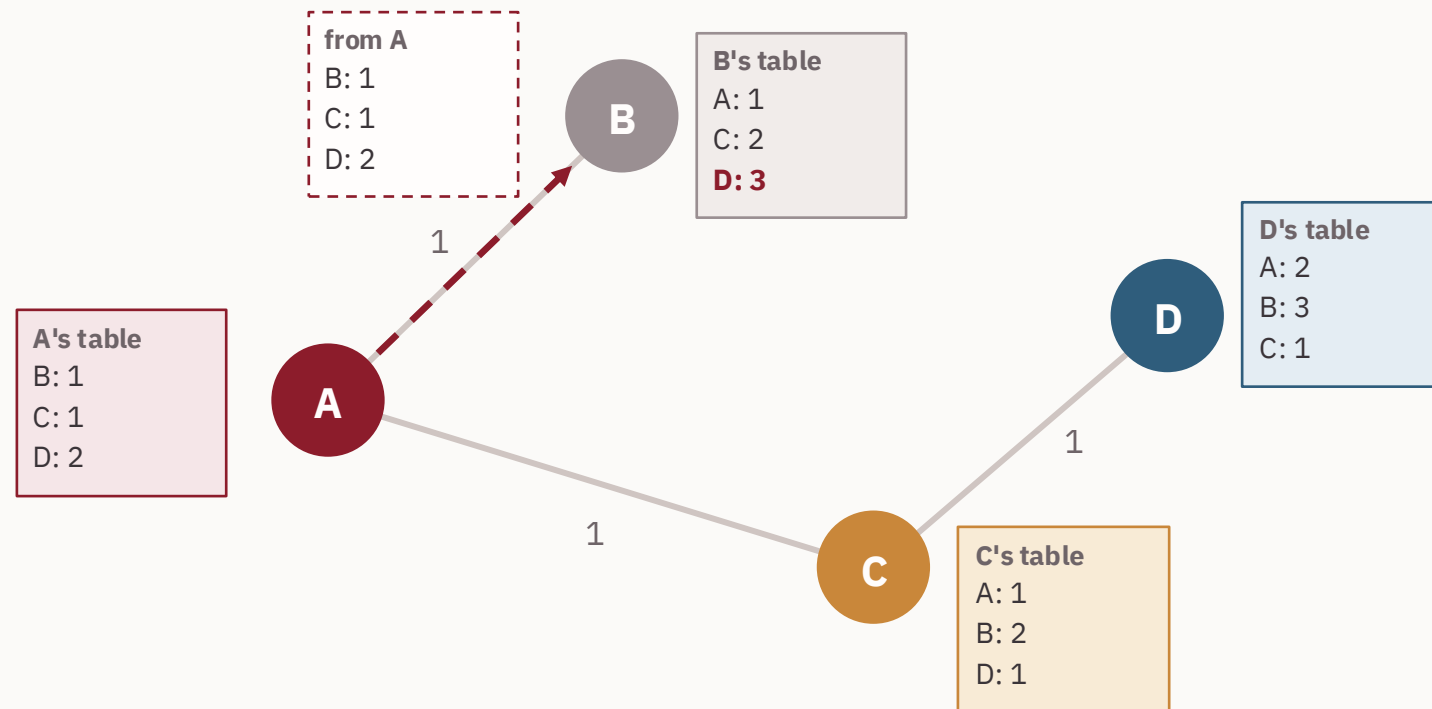
WHAT HAPPENS

- **A learns D at 2:** through C, $1 + 1 = 2$
- **D learns A at 2 and B at 3** through C
- **A keeps B at 1:** the route through C would cost 3, a worse metric, so it is discarded

LEGEND

-  table of the node
-  vector just received
- 1** entry updated in this step

DSDV: the news reaches B



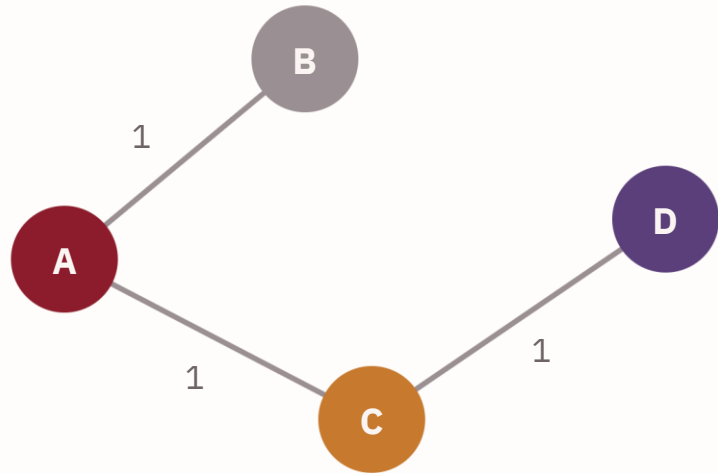
WHAT HAPPENS

- **A has a new route, so it sends** its updated vector (B: 1, C: 1, D: 2) to B
- **B learns D at 3:** through A, $1 + 2 = 3$
- **Converged:** every node has a route to every destination; the final tables are on the next slide

LEGEND

-  table of the node
-  vector just received
- 1** entry updated in this step

DSDV: distance vectors at work



HOPS AFTER CONVERGENCE

Table of	to A	to B	to C	to D
A	–	1	1	2
B	1	–	2	3
C	1	2	–	1
D	2	3	1	–

1. Start: neighbors only, ∞ elsewhere
2. First exchange: two-hop routes (A–D, B–C)
3. Second exchange: three-hop route B–D

Reactive routing: routes on demand

Discovery only when needed

A source that needs a route starts a network-wide search

No constant updates

Much less overhead, but each discovery adds delay

Route cache

A cached route avoids the delay; discovery ends when a route is found or all options are tried

NO ROUTES AT ALL

Flooding · Gossiping

ROUTE DISCOVERY

DSR · AODV

Flooding and gossiping

Flooding

Forward every packet on every link except the one it came from. No topology maintenance, no route discovery.

Pro · delivery guaranteed if a route exists

Pro · one copy arrives by the quickest route

Drawbacks

Implosion · duplicate copies reach the same node

Overlap · nodes covering the same region send the same data

Resource blindness · ignores the energy nodes have left

Gossiping

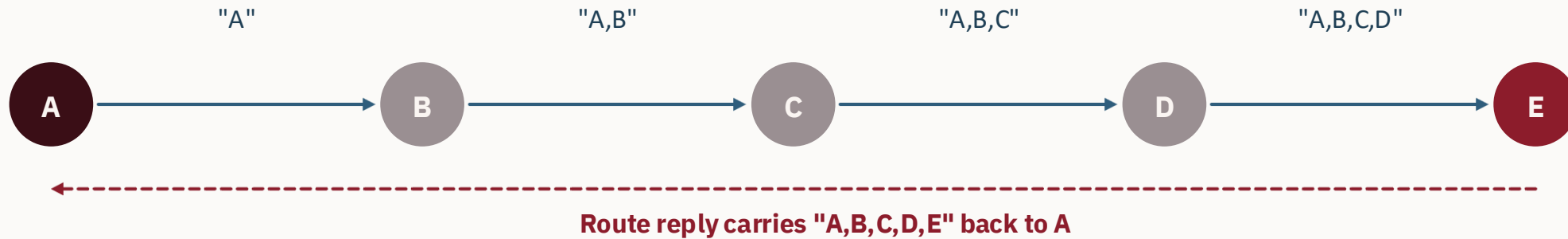
Forward to **one random** neighbor only.

Pro · one copy per node: no implosion

Con · slow to reach every node

Every node in range receives and forwards every packet: an enormous amount of traffic, and a **short network lifetime**.

Dynamic Source Routing (DSR, RFC 4728)



Source routing

Each packet carries the full list of hops; nodes that forward or overhear it can cache the route

Route discovery

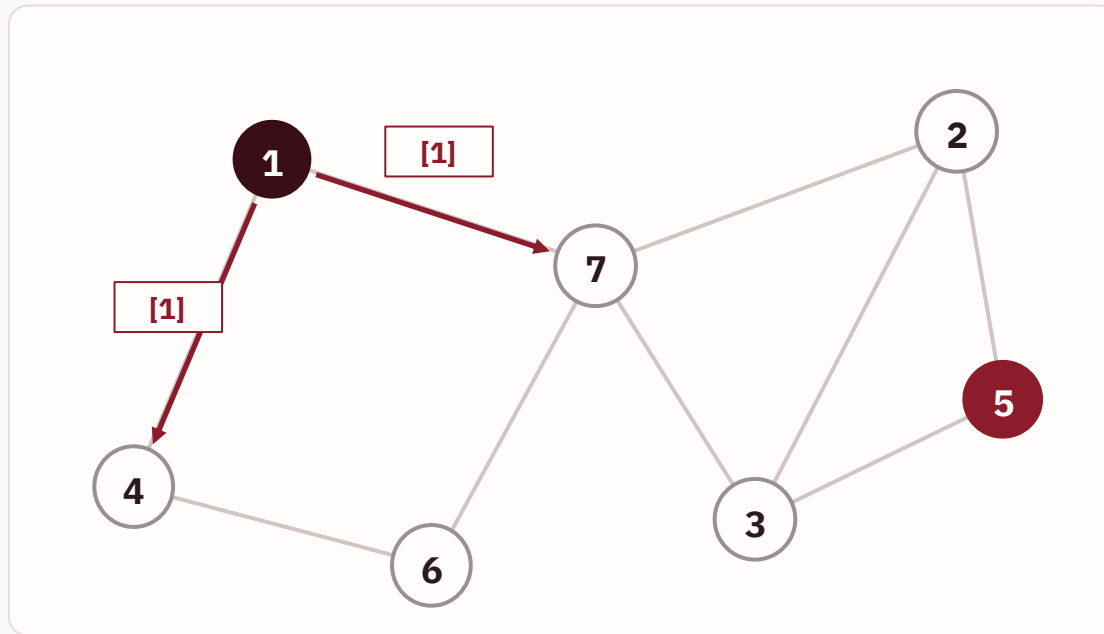
Route Request with a unique ID; each node appends its address; the target replies with the record

Route maintenance

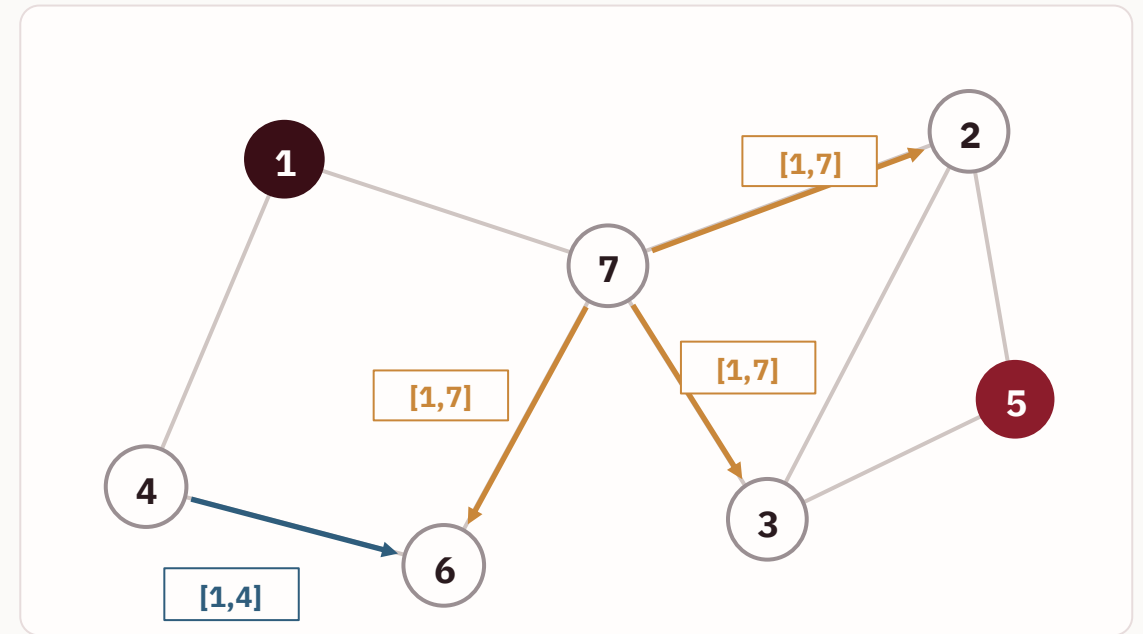
On a broken link, try another cached route or discover again; multiple routes speed this up

DSR route discovery: the request spreads

STEP 1 · NODE 1 LOOKS FOR A ROUTE TO 5: IT BROADCASTS A RREQ



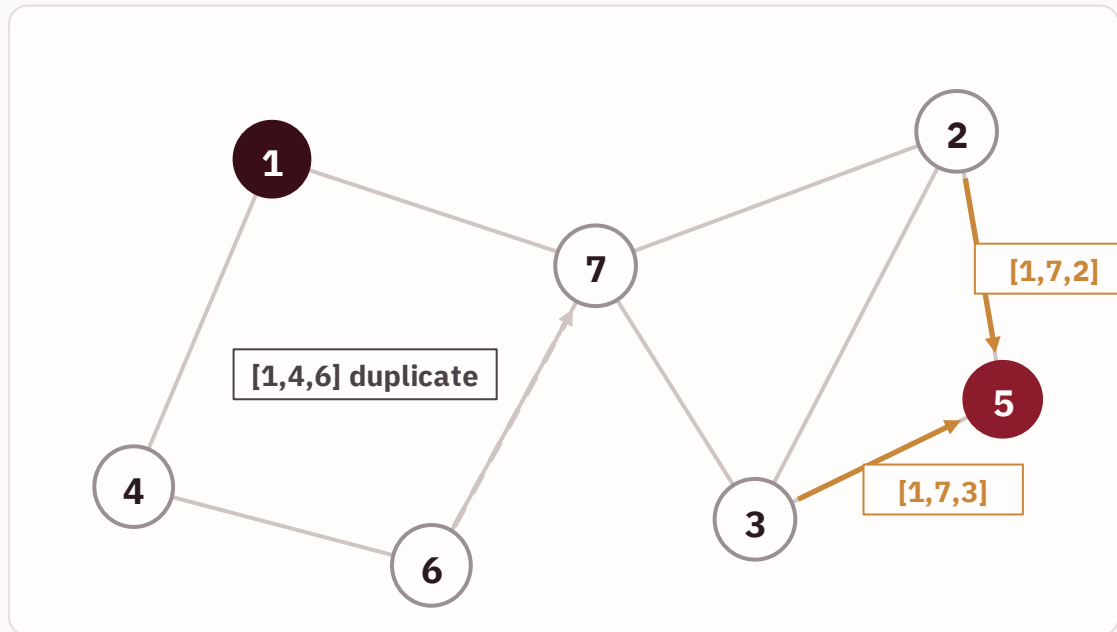
STEP 2 · NEIGHBOURS APPEND THEIR ADDRESS AND REBROADCAST



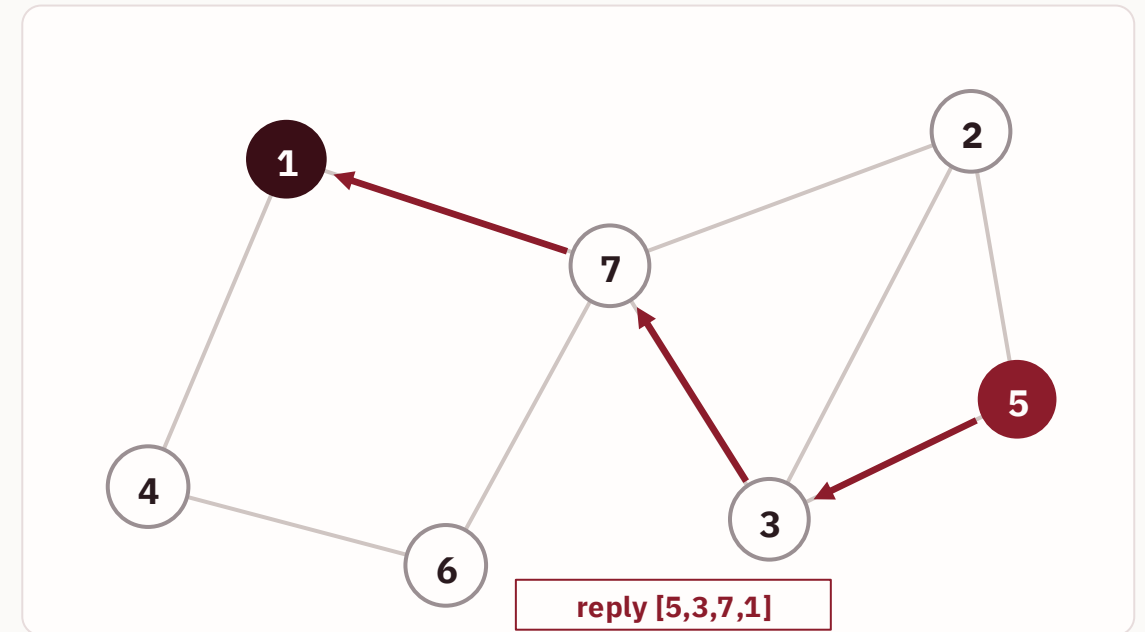
Each Route Request contains a unique request ID and a record of the addresses of the intermediate nodes this copy has been forwarded through.

DSR route discovery: the reply comes back

STEP 3 · COPIES OF THE RREQ REACH THE TARGET



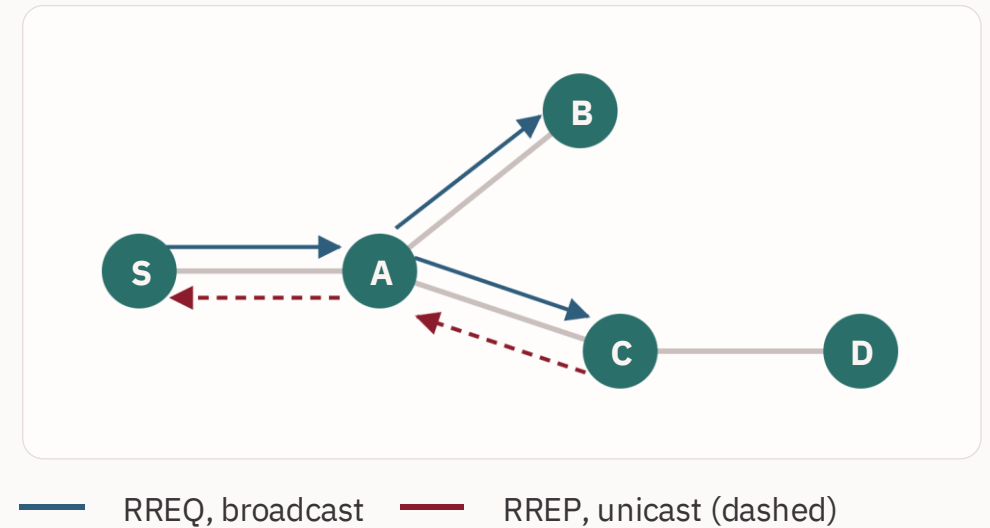
STEP 4 · 5 REPLIES, BY SOURCE ROUTING



Node 5 uses the route recorded in the RREQ to send back, via **source routing**, a route reply [5,3,7,1]. Node 7 discards the copy that arrives again with the same request ID.

AODV: on-demand distance vector

1. S broadcasts a **RREQ** (source S, destination D)
2. A stores the reverse route "to S via S", rebroadcasts
3. C stores the reverse route "to S via A"
4. C has a fresh enough route to D: unicasts a **RREP** to A
5. A stores the forward route "to D via C", sends the RREP to S
6. S stores "to D via A": the route is ready



Like DSR, but with **next-hop routing tables** instead of source routes and **sequence numbers** against stale routes; unlike DSDV, routes exist only when needed.

Geographic routing

SO FAR

Explicit routing tables

Built and maintained by the protocol to say which next hop to use



ALTERNATIVE

Infer it from positions

Knowing my position, my neighbors' and the destination's: send to a neighbor in the right direction

Geocasting

Deliver to any node in a given area

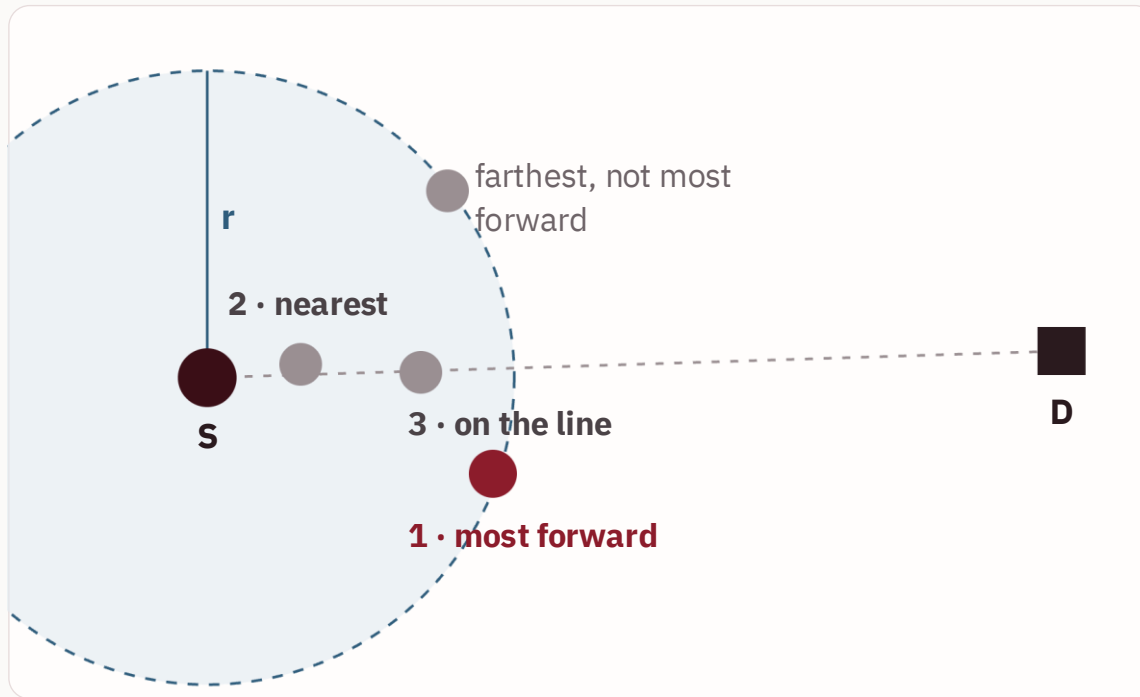
Position-based routing

Positions guide the route to one node

Location service

May be needed to map a node ID to its position

Choosing the next hop



1 • Most forward within r

Most progress towards **D**, not the farthest from **S**

2 • Nearest with forward progress

Short hops: minimize transmission power

3 • Directional

Angularly closest to **D**, or closest to the line **S-D**; may loop

Dead ends

A node may have no neighbor closer to **D**: simple strategies get stuck

ROUTING

Which approach is best?

Approach	Strength	Cost
Proactive	Fast: the route is already there	Constant update overhead
Reactive	Much less overhead	Slower: discovery delay
Geographic	Saves the most energy, scales	Needs positions; dead ends

It depends on the network: **mobility** and **node capabilities**.

FOR DISCUSSION

A network of drones flying over an area: the topology changes continuously and nodes are often disconnected. Which approach would you choose?

WRAP-UP

Four things to remember

- 1 The opening question: spend energy on useful bits, not on listening to an idle channel.
- 2 S-MAC = CSMA/CA collision avoidance + coordinated sleep, overhearing avoidance, message passing.
- 3 It trades latency and fairness for energy; the frame length is the knob.
- 4 Routing: proactive is fast but costly, reactive is cheap but slow, geographic saves energy when positions are known.

REFERENCES

Further reading

- **W. Ye, J. Heidemann, D. Estrin**, "An Energy-Efficient MAC Protocol for Wireless Sensor Networks", IEEE INFOCOM 2002
- **W. Ye, J. Heidemann, D. Estrin**, "Medium Access Control With Coordinated Adaptive Sleeping for WSNs", IEEE/ACM ToN 12(3), 2004
- I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, E. Cayirci, "Wireless Sensor Networks: A Survey", Computer Networks 38(4), 2002